Understanding Denoising Diffusion Probabilistic Models

As you may already know about a sub field of machine learning, called generative ML where we try to learn the underlying distribution fo the data and then try to generate data from that learned distribution. It has multiple applications, like generating synthetic data, creating deep fakes etc (mention other applications). There are multiple ways to do this including GANs, Encoder decoder, DDPMs etc. Today we are gonna discuss one of them in detail, i.e., Denoising diffusion models.

If i have to say simply, DDPMs is just learning to remove the noise from a sample. For example, you can take an image and we start adding noise to it in hundreds of small small steps, until the sample turns totally into noise. On each step we train a model to identify the noise, there are minor variations based on methods, for examples:
1. Train model to identify the noise added at that step (xT - xT-1)
2. Train model to directly get the denoised image from noise added image (xT-1)
3. I think there is one more, but if not -> then remove this.

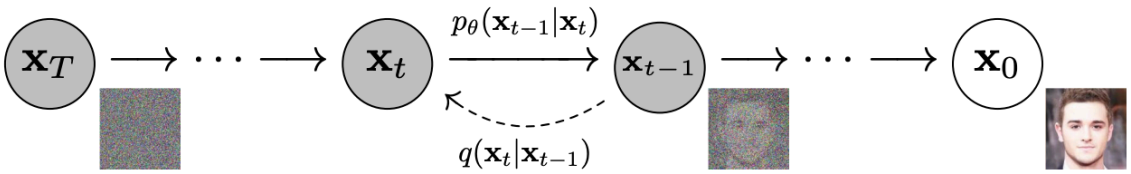


Figure 2: The directed graphical model considered in this work.

Also this noise adding procedure is considered a markov chain, as it has markov property the way it is defined in this paper.

During sampling or inference, we just try to reverse this process, given a total noise sample, keep removing noise and eventually you get a new data point from the same distribution. Some examples are shown below in the figure.

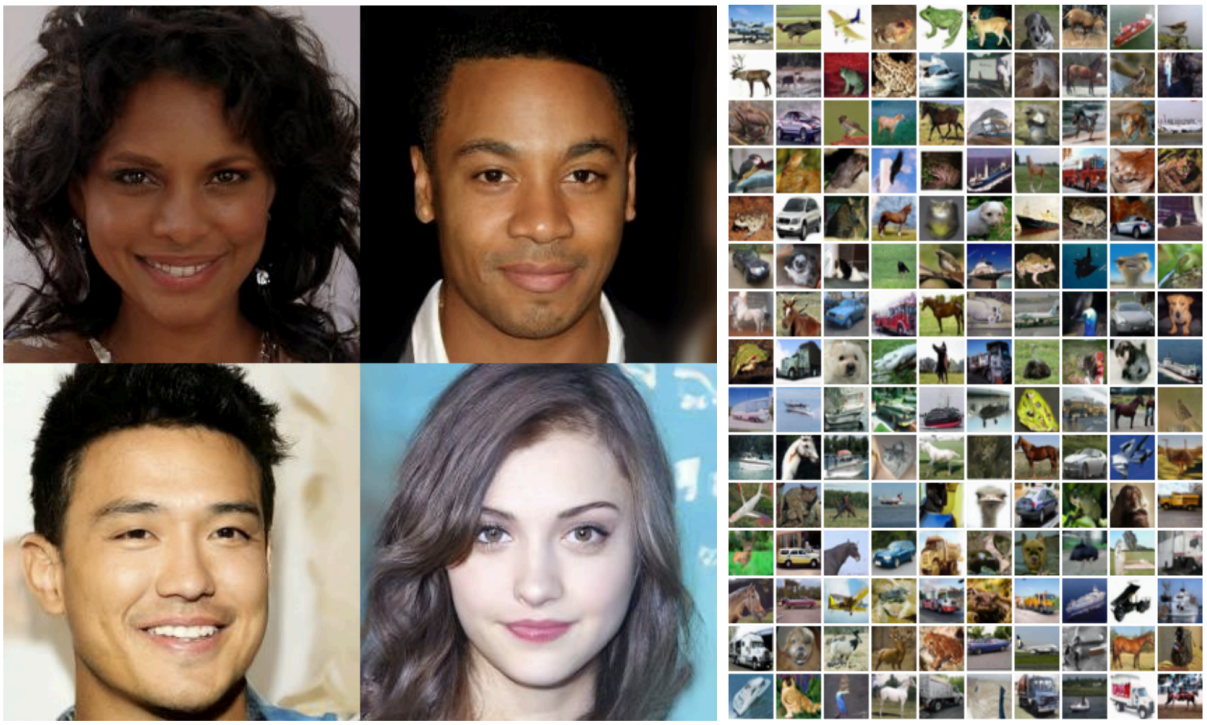


Figure 1: Generated samples on CelebA-HQ 256×256 (left) and unconditional CIFAR10 (right)

Now going deeper into the training, let's discuss a bit about the loss. What kind of loss we would like to use to train such a network?

Training is performed by optimizing the usual variational bound on negative log likelihood:

$$\mathbb{E}\left[-\log p_\theta(\mathbf{x}_0)\right] \leq \mathbb{E}_q\left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] = \mathbb{E}_q\left[-\log p(\mathbf{x}_T) - \sum_{t\geq 1} \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})}\right] =: L \quad (3)$$

The loss reduces to:

$$\mathbb{E}_q\left[\underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\mathrm{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0}\right] \quad (5)$$

If you look closely at equation 5, there are 3 terms, each serves a different purpose, i will try to explain them as intuitively as possible:
1. LT -> After T steps, the given sample is actually reduced to noise. <I am not 100% sure here, if i am wrong or it can be explained, please elaborate and make it better>
2. LT-1 -> <i don't remember please complete it based of your understanding.>. If i have to understand, it has something to do with if we are able to predict the noise correctly or not. And
3. L0 -> is simply reconstruction loss.

On rearranging stuff, everything simply reduces to:

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

The gradient step is simply predicting the noise (loss function).

A couple of question you might have is what is alphat and betat etc. Beta t is called noise schedule, the idea is simple, we want to make the final distribution standard gaussian, which means it should have a mean of 0 and variance of one. So since we know if we add random variables sampled from gaussian of mean 0, their summed distribution will also have mean of zero, similarly we decide the variance same way. Mean is sorted, but the variance needs to be scaled appropriately, hence the noise schedule.

Everything else is just implementation details, you can find my implementation on MNIST dataset here: https://github.com/natsu-git-hub/ddpm

Resources:
Cite the paper here.